

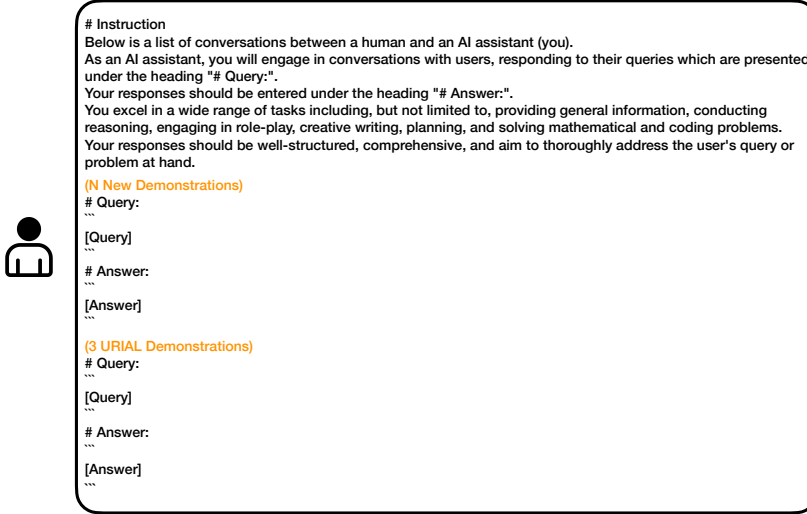


Figure 8: The prompt template for doing ICL in our work. Note that the words in orange are solely for illustration purposes and do not appear in the real prompts. The mixture of new demonstrations we test in our experiments is inserted before URIAL demonstrations.

demonstration, to the original 3-shot URIAL prompt. We evaluate the resulting instruction-following performance for each prompt on MT-Bench but with GPT-4-Turbo as the judge since GPT-4-Turbo is cheaper and faster than GPT-4 (the original LLM judge for MT-Bench evaluation), and also has a high correlation with human judgment. Finally, the example with the highest 1st-turn MT-Bench score is chosen.

We continue our greedy search for the proper fifth and sixth demonstration in a more restricted search space due to the heavy computational cost of the search process. We keep reducing the search space by applying a threshold of 1st-turn MT-Bench score (i.e., 6.2 in our experiments). Similarly, we add each example from the new search space on top of the best (N-1)-shot prompt found in previous steps and generate a set of new N-shot prompts. Then we run the MT-bench evaluation with GPT-4-Turbo as the judge for multiple times and select the best (N+1)-shot prompt.

A.4 SCALING EXPERIMENTS

The recently released LLMs with increasingly large context windows allow using many more shots for ICL than are typically used. Prior works [Agarwal et al. \(2024\)](#); [Bertsch et al. \(2024\)](#) have shown that many-shot ICL, compared to few-shot ICL, can make task-specific fine-tuning less essential and allows LLMs to tackle a wider range of tasks without specification. Therefore, we want to examine if the instruction-following performance of base LLMs could benefit from many-shot ICL through various scaling experiments, and systematically compare ICL with IFT. In the following, we add more details about these scaling experiments, including the prompt template, data selection for both ICL and IFT, and training hyper-parameters for IFT experiments.

ICL Random. We construct the set of in-context demonstrations based on the high-quality data we select from SkillMix-4k or Evol-Instruct-70k dataset. Through randomly sampling from the high-quality dataset for multiple times, we generate a series of in-context demonstration sets that each contains N examples, where $N \in \{0, 7, 17, 27, 37, 47\}$ for Mistral-7B-v0.2 model (32k context length) and $N \in \{0, 7, 17, 27, 37, 47, 97, 147, 197, 247\}$ for Llama-3.1-8B model (128k context length), and insert the N demonstrations into the prompt template for ICL as shown in Fig 8. Note that the prompt template is strictly following the one used in URIAL paper and we only replace the demonstrations used for the ICL purpose. The average performance and standard deviations are computed over 5 random seeds.

```

# Instruction
Below is a list of conversations between a human and an AI assistant (you).
As an AI assistant, you will engage in conversations with users, responding to their queries which are presented under the heading "# Query:".
Your responses should be entered under the heading "# Answer:".
You excel in a wide range of tasks including, but not limited to, providing general information, conducting reasoning, engaging in role-play, creative writing,
planning, and solving mathematical and coding problems.
Your responses should be well-structured, comprehensive, and aim to thoroughly address the user's query or problem at hand.

# Query:
...
Can you tell me some common types of renewable energy sources?

# Answer:
Hello! Certainly! Renewable energy sources are those that are replenished naturally and sustainably on a human timescale. Here are some common types of renewable energy sources:

1. Solar Energy: Energy harnessed from the sun. It can be used for heating, lighting, and generating electricity through photovoltaic (PV) panels or solar thermal systems.
2. Wind Energy: Energy generated from the natural movement of air. Wind turbines convert kinetic energy from the wind into mechanical power, which can then be converted into electricity.
3. Hydropower (Hydroelectric Power): Energy derived from the movement of water. This can be the flow of rivers, tidal movements, or wave power. Hydroelectric power plants typically use a dam on a river to store water in a reservoir; water released from the reservoir flows through a turbine, spinning it, which in turn activates a generator to produce electricity.
4. Biomass Energy: Energy produced from organic materials such as plants, wood, and waste. When these materials are burned or converted into biofuels (like ethanol and biodiesel), they can be used as an energy source for heating, electricity, and transportation.
5. Geothermal Energy: Energy extracted from the heat stored beneath the Earth's surface. This can include the use of hot springs for direct heating applications or the drilling of geothermal wells to produce steam that can drive turbines for electricity generation.

Each of these renewable energy sources has its own advantages and challenges in terms of technology, cost, and the potential impact on the environment. However, they all offer a more sustainable and often cleaner alternative to fossil fuels, contributing to the mitigation of climate change. Please let me know if you have any other questions!

# Query:
...
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

# Answer:
...
Hi, I'm happy to help you. To solve the problem, let's think step by step:

1. In April, Natalia sold clips to 48 friends.
2. In May, she sold half as many as she did in April. So, the number of clips she sold in May =  $48/2 = 24$ .
3. To find out how many clips she sold altogether in April and May, simply add the number of clips from both months:  $48 \text{ (April)} + 24 \text{ (May)} = 72$  clips.

Therefore, Natalia sold 72 clips altogether in April and May. Please feel free to ask me if you have any more questions!

# Query:
...
What is the proper method for deep frying a turkey?

# Answer:
Hello! I'm happy to help you with your question about deep frying a turkey! Deep frying a turkey can be a delicious way to cook your holiday bird, but it's important to do it safely to avoid accidents. Here's a step-by-step guide to deep frying a turkey:

1. Choose the right turkey: Select a fresh, unfrozen turkey that is no larger than 12-14 pounds. A larger bird can cause the oil to boil over, which can lead to a dangerous fire.
2. Prepare the turkey: Thoroughly thaw and dry the turkey before frying. Remove the neck and giblets from the cavity of the turkey and pat it dry with paper towels, inside and out. This will help prevent oil splatter and ensure a crispy skin.
3. Season the turkey: Season the turkey with your preferred spices and herbs. You can use a dry rub or inject the seasoning directly into the meat.
4. Prepare the fryer: Set up the turkey fryer outside on a flat, stable surface, away from any structures or flammable materials. Fill the fryer with peanut or canola oil to the 1. recommended level, typically indicated on the fryer. Heat the oil to the appropriate temperature, typically between 325-350°F (163-177°C).
5. Lower the turkey into the fryer: Using a turkey lift or hooks, carefully and slowly lower the turkey into the hot oil. Make sure the turkey is fully submerged in the oil.
6. Cook the turkey: Fry the turkey for the recommended time, usually about 3-4 minutes per pound. Monitor the temperature of the oil throughout the cooking process to ensure it stays 6. within the recommended range.
7. Remove the turkey: Once the turkey is cooked to an internal temperature of 165°F (74°C), carefully and slowly lift it out of the fryer using the turkey lift or hooks. Place it on a wire rack or tray to drain any excess oil.
8. Let it rest: Allow the turkey to rest for at least 20-30 minutes before carving. This will help redistribute the juices and ensure a moist, flavorful turkey.

Remember to always prioritize safety when deep frying a turkey. Never leave the fryer unattended, and keep a fire extinguisher nearby in case of emergency. Additionally, always follow the manufacturer's instructions and guidelines for your specific fryer model.

```

Figure 9: **The details of URIAL prompt.** It consists of a carefully designed set of rules after “# Instruction” and three curated stylistic examples, each of which always begins with affirming the user query by saying something similar to “Hello! I’m happy to help you.”.

ICL Random (no URIAL). We generate a series of in-context demonstration sets by randomly sampling from the high-quality IFT dataset, and each set contains N examples, where $N \in \{3, 10, 20, 30, 40, 50\}$ for Mistral-7B-v0.2 model (32k context length) and $N \in \{3, 10, 20, 30, 40, 50, 100, 150, 200, 250\}$ for Llama-3.1-8B model (128k context length). URIAL examples will not be added to the context, so it ensures the total number of in-context examples are the same as the Random group. The average performance and standard deviations are computed over 5 random seeds.

ICL Greedy Search. Following the procedure we mention in Sec. A.3, ideally we can get as many optimal examples as we want if we have sufficient OpenAI API credits. Thus it allows us to create another series of in-context demonstration sets by selecting another N examples for each $N \in \{1, 2, 3\}$ in the paper. The resulting mixture of in-context demonstrations is then placed in the corresponding location of the prompt template as shown in Fig. 8.

Table 5: Details of training hyperparameters for IFT experiments.

Data Size	# GPUs	Epochs	LR	LR Scheduler	Batch Size	Context Win. Len.	WD	Warmup Rate
Mistral-7B-v0.2								
3	2	6	2e-6	Cosine	2	2048	0.01	0.03
10	4	6	2e-6	Cosine	8	2048	0.01	0.03
20	4	6	2e-6	Cosine	8	2048	0.01	0.03
30	4	6	2e-6	Cosine	8	2048	0.01	0.03
40	4	6	2e-6	Cosine	8	2048	0.01	0.03
50	4	6	2e-6	Cosine	8	2048	0.01	0.03
100	4	6	2e-6	Cosine	8	2048	0.01	0.03
300	4	6	2e-6	Cosine	8	2048	0.01	0.03
1000	4	15	2e-6	Cosine	128	2048	0.01	0.03
2000	4	15	2e-6	Cosine	128	2048	0.01	0.03
4000	4	15	2e-6	Cosine	128	2048	0.01	0.03
Llama-3.1-8B								
3	2	6	4e-6	Cosine	2	2048	0.01	0.03
10	4	6	4e-6	Cosine	8	2048	0.01	0.03
20	4	6	4e-6	Cosine	8	2048	0.01	0.03
30	4	6	4e-6	Cosine	8	2048	0.01	0.03
40	4	6	4e-6	Cosine	8	2048	0.01	0.03
50	4	6	4e-6	Cosine	8	2048	0.01	0.03
100	4	6	4e-6	Cosine	8	2048	0.01	0.03
300	4	6	4e-6	Cosine	8	2048	0.01	0.03
1000	4	15	4e-6	Cosine	128	2048	0.01	0.03
2000	4	15	4e-6	Cosine	128	2048	0.01	0.03
4000	4	15	4e-6	Cosine	128	2048	0.01	0.03

Table 6: **Improved in-context demonstrations selected from Evol-Instruct-70k dataset for Mistral-7B-v0.2 base model.** We report the 1st-turn score on MT-Bench and the length-controlled (LC) win rates on AlpacaEval 2.0 when using different IC prompts.

Model	MT-Bench (1st)	AlpacaEval 2.0
URIAL (3 examples)	6.99	8.09
URIAL + greedy search (1 ex.)	7.46	7.91
URIAL + greedy search (2 ex.)	7.69	8.38
URIAL + greedy search (3 ex.)	<u>7.68</u>	9.22

IFT. We run instruction fine-tuning the base LLMs (Mistral-7B-v0.2 and Llama-3.1-8B) either with tens of examples (i.e., few-sample regime) or thousands of examples. The training examples are randomly sampled from existing IFT datasets, SkillMix-4k and Evol-Instruct-70k. The mean score and standard deviations are calculated over multiple random seeds. More specifically, we use 5 random seeds for Mistral-7B-v0.2 model and 3 random seeds for Llama-3.1-8B model due to a restricted compute budget. In particular, since the size of SkillMix dataset is 4k, the randomness of IFT with 4k examples primarily comes from optimization process and scoring evaluation with GPT-4, otherwise part of randomness also comes from data sampling process. We list the details of training hyper-parameters used in IFT experiments in Table. 5. We always select the last model checkpoint to run evaluation.

B REVISIT OUR FINDINGS ON INSTRUCT MODELS

In this section, we want to revisit some of our key findings in the paper on instruct model, including influence of the individual components of URIAL (§ B.1), importance of question-answer matching of demonstrations for in-context alignment (§ B.2), and many-shot in-context alignment (§ B.3). We experiment using Mistral-7B-Instruct-v0.2, the instruct version of Mistral-7B-v0.2 that is widely used in the main paper, and again we adopt MT-Bench with GPT-4 as the judge for evaluation.

Table 7: **Influence of the individual components of URIAL on instruct model** We report the mean MT-Bench scores for cases that have multiple different combinations, such as Rule + 1 example and Rule + 2 example. The standard deviation is always calculated on 3 runs.

In-context prompt	1st-turn	2nd-turn	Average
Empty	7.92 $\pm$ 0.08	7.18 $\pm$ 0.05	7.55 $\pm$ 0.06
Rule-only	7.80 $\pm$ 0.07	7.15 $\pm$ 0.10	7.48 $\pm$ 0.09
Rule + 1 example	7.54 $\pm$ 0.15	6.82 $\pm$ 0.34	7.18 $\pm$ 0.19
Rule + 2 examples	7.70 $\pm$ 0.17	6.84 $\pm$ 0.21	7.27 $\pm$ 0.19
URIAL	7.58 $\pm$ 0.06	6.48 $\pm$ 0.04	7.21 $\pm$ 0.06

Table 8: **Importance of question-answer matching of demonstrations for in-context alignment on instruct models.**

In-context prompt	1st-turn	2nd-turn	Average
no demonstrations	7.92	7.18	7.55
X only (no Y)	4.29	6.06	5.18
Y only (no X)	7.53	<u>6.85</u>	7.19
circular shift of Y	6.09	5.99	6.04
in-domain random Y	6.62	5.94	6.28
out-of-domain random	5.69	5.50	5.60
correct Y	<u>7.58</u>	6.48	<u>7.21</u>

B.1 INFLUENCE OF THE INDIVIDUAL COMPONENTS OF URIAL ON INSTRUCT MODEL

In Table 7, we find that different from what we observed from the base LLM, adding in-context examples, even only rules, weakens the instruction-following performance of instruct models as measured by MT-Bench performance, and more examples lead to more performance declines.

B.2 IMPORTANCE OF QUESTION-ANSWER MATCHING OF DEMONSTRATIONS FOR IN-CONTEXT ALIGNMENT ON INSTRUCT MODELS

In Table 8, we find that adding examples in the context consistently causes damage to the instruction-following performance of instruct models, even when using correct URIAL examples, which is similar to what we observe on the instruct model in Table 7. Surprisingly, it leads to a pretty bad 1st-turn score (4.29) on MT-Bench when only presenting queries and no corresponding answers in the context. After looking into the model responses, we find that the instruction model sometimes would manage to answer all queries present in the context, including the ones as demonstrations, and outputs consisting of answers to multiple questions lead to low performance on model-based automatic evaluations. This unexpected behavior is mitigated in the 2nd-turn evaluation as a complete query-answer pair is added to the context after the 1st-turn answer generation.

B.3 MANY-SHOT IN-CONTEXT ALIGNMENT ON INSTRUCT MODEL

We further test the effect of alignment with ICL on Mistral-7B-Instruct-v0.2 when increasing the number of demonstrations. We test on the same data we used in Fig. 3 and more details can be found in App. A.4. The results of many-shot in-context alignment experiments are shown in Fig. 10. We find that alignment through ICL does not further improve the instruction-following performance of instruct models, but slightly declines its capability to answer user queries.

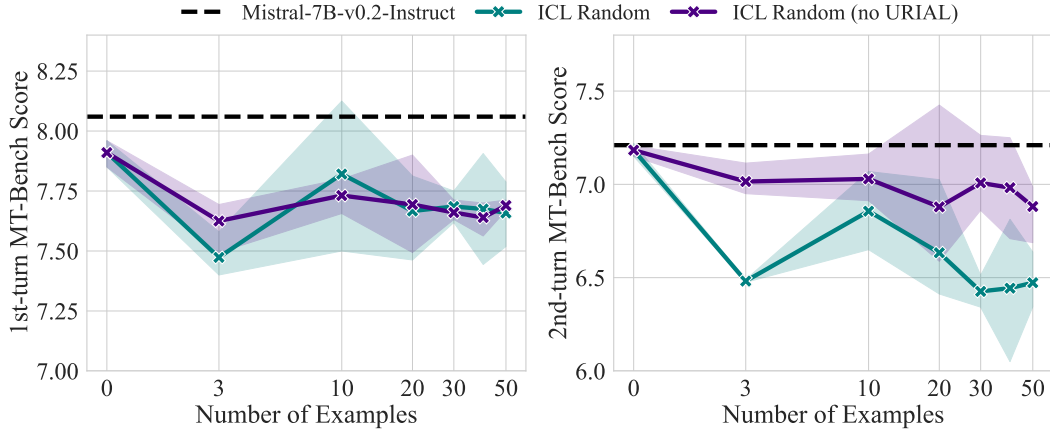


Figure 10: **Scaling the number of demonstrations for alignment with ICL on Mistral-7B-Instruct-v0.2.** We measure the alignment performance of different settings using the MT-Bench score. The base model is always Mistral-7B-Instruct-v0.2 for ICL Random and ICL Random (no URIAL).

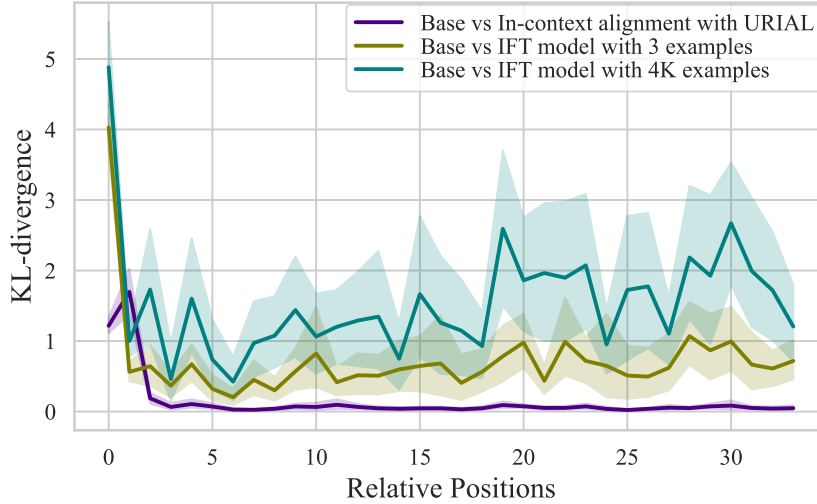


Figure 11: **KL-divergence between the base LLM and its aligned model using ICL or IFT.** The base model is always Mistral-7B-v0.2. The KL-divergence results are averaged over 25 examples.

C KL-DIVERGENCE BETWEEN THE BASE LLM AND ITS ALIGNED MODEL USING ICL OR IFT

Since we always experiment with open source LLMs with 7 ~ 8 billions parameters, we are interested in mechanistic study with logits of model outputs to reveal more insights, such as the KL-divergence between the base LLM and its aligned counterparts.

Experimental setup. We randomly select 25 query-response paired examples from SkillMix dataset as the data to calculate KL-divergence between the base LLM and the aligned models. Specifically, for each query-response paired example, two models predict the next token given the same input respectively. The model only predicts tokens from responses and each answer token after calculating KL-divergence will be added to the input for the next prediction. The response from each example is truncated to save computation costs. We always select Mistral-7B-v0.2 as the base model, and another 3 aligned models, in-context alignment using URIAL, IFT with 3 SkillMix examples, and IFT with 4k SkillMix examples, as target models to calculate KL-divergence.

Results. From Fig. 11, we observe that the probability distribution of the next token prediction, for the in-context aligned model, differs significantly only for the first few tokens, after which the